

# Deep RL for Automated Stock Trading

CS5242 Neural Networks and Deep Learning — Group ID24

Yufan Zhu   Rutwik Shrikant Dharanappagoudar   Xin Fei

Semester 2, 2025/26

## Abstract

We design and implement Deep Q-Network (DQN) and Double DQN (DDQN) entirely from scratch in PyTorch and apply them to a single-asset stock-trading task on AAPL drawn from daily DJIA OHLCV data over 2009 to 2020. Our goal is explicitly pedagogical. We aim to understand what each component of a textbook DQN pipeline actually does when stressed by real market data, rather than to beat state-of-the-art trading systems. These components include the replay buffer, the target network, the exponential  $\epsilon$ -decay schedule, reward clipping, the Huber loss and the Double-DQN target decoupling. We build a Gym-style `TradingEnv` with a 9-feature technical-indicator state, a three-action position-delta action space and a clipped log-return reward with an explicit drawdown penalty. On a 437-day out-of-sample test split spanning the COVID-19 crash of early 2020, DQN learns a non-trivial policy that achieves a cumulative return of +49.6% and a Sharpe of 0.84, while DDQN adopts a visibly more cautious policy and ends near flat. Neither agent surpasses a passive buy-and-hold benchmark on this strongly trending asset. We benchmark both agents against four classical baselines of increasing sophistication and find that a simple 20-day price momentum rule already beats buy-and-hold in cumulative return, Sharpe and maximum drawdown, while a Random Forest trained on the same feature set performs substantially worse than the DQN agent. The exercise makes the role of each DQN ingredient concretely visible and directly addresses the course learning goal of understanding DRL from first principles.

## 1 Introduction

Stock trading is naturally framed as a sequential decision-making problem in which an agent observes market state and chooses to buy, hold or sell so as to maximise long-term profit, exactly the setting targeted by Reinforcement Learning [1]. Classical rule-based trading strategies are brittle and struggle to adapt to non-stationary market dynamics, which motivates a learning-based approach.

Deep Reinforcement Learning (DRL) [2, 1] learns trading policies directly from data without hand-crafted rules, and a now-substantial line of work has applied it to financial decision making [9, 10, 11, 12]. We implement DQN [2] and Double DQN (DDQN) [3] entirely from scratch in PyTorch [8] and evaluate them on historical DJIA data with AAPL as a single-asset case study. We compare against four classical baselines of increasing sophistication, namely passive buy-and-hold, an ARIMA(1,0,0) walk-forward forecaster, 20-day price momentum and a Random Forest classifier trained on the same feature set as the RL agents.

Our goal is explicitly pedagogical. We aim to understand DRL by designing and analysing these algorithms from first principles on a realistic financial task. We do not aim to beat the passive benchmark. We aim to see clearly where a textbook DQN agent succeeds, where it fails and why, and to place its behaviour in context against simple classical rules on the same data.

## 2 Data

We use the `alincijov/trading` Kaggle dataset, which provides daily OHLCV data for 30 DJIA stocks from 2009 to 2020, roughly 2,900 days per ticker, downloaded programmatically via `kagglehub`. Our data pipeline loads a 10-ticker subset of the DJIA, and all experiments in this report focus on AAPL as a single-asset case study. The pipeline also supports other tickers for future multi-asset extensions. Data is split chronologically into 70%, 15% and 15% train, validation and test sets, and train statistics are used for feature normalisation so as to avoid look-ahead bias.

For each ticker we compute 9 features. These are the daily return, two price deviations from short and long moving averages, RSI-14, the MACD signal, Bollinger Band width, normalised volume, the Average True Range (ATR) and the On-Balance Volume (OBV). All features are z-score normalised using training-set statistics only to prevent data leakage. Raw returns and close prices are preserved separately for reward and portfolio computation.

Figure 2 shows the Pearson correlation between all 9 features on AAPL. Most pairs exhibit low correlation, which suggests limited linear redundancy among these indicators on this ticker. The strongest correlations are between Bollinger Band width and ATR at 0.63, which is expected since both measure volatility, and between the two moving-average deviations at 0.45. We treat this as a single-ticker observation rather than a general claim.

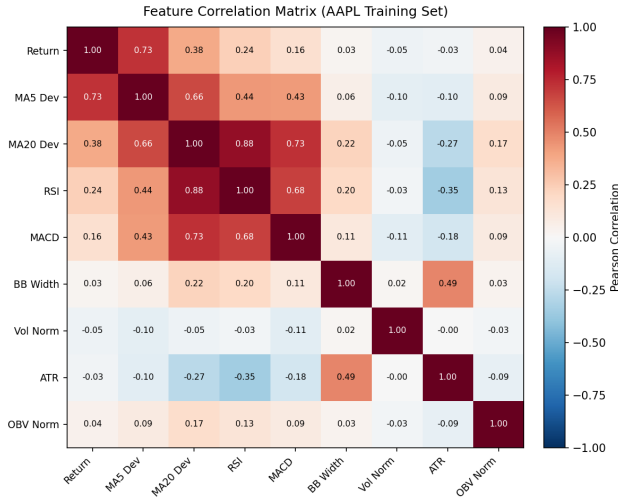


Figure 2: Feature correlation matrix on the AAPL training set. Most pairs show low linear correlation on this ticker, suggesting limited linear redundancy.

The full technical indicator time series and feature distribution comparisons across splits are provided in Appendix A (Figures 5 and 6).

### 3 Environment Design

We implement a Gym-style [7] TradingEnv for single-asset trading. The state  $s_t \in \mathbb{R}^{10}$  concatenates the 9 normalised features with the current integer position  $\text{pos}_t \in \{-1, 0, +1\}$ , where the three values correspond to short, flat and long. The position is embedded

as an additional scalar entry in the state vector. The agent chooses one of three discrete actions that apply a delta to the position, namely Decrease, Hold and Increase with deltas  $-1$ ,  $0$  and  $+1$ . The resulting position is clamped to  $[\text{pos}_{\min}, \text{pos}_{\max}] = [-1, +1]$ , so a flat agent cannot jump directly from short to long. This prevents abrupt reversals and makes partial transitions natural. Shorting is allowed throughout training and evaluation.

The reward is a clipped log-return with an explicit drawdown penalty. We first update the portfolio according to the chosen position,

$$V_t = V_{t-1} (1 + \text{pos}_t \cdot \tilde{r}_t - c_t \cdot \mathbf{1}[\text{pos}_t \neq \text{pos}_{t-1}]), \quad (1)$$

where  $\tilde{r}_t$  is the raw un-normalised daily return and  $c_t$  is a transaction cost applied only when the position changes. We then define

$$r_t = \log \frac{V_t}{V_{t-1}} + \lambda \min\left(0, \frac{V_t - V_t^{\text{peak}}}{V_t^{\text{peak}}}\right),$$

$$r_t \leftarrow \text{clip}(r_t, -1, +1), \quad (2)$$

with  $\lambda = 0.5$  and  $V_t^{\text{peak}} = \max_{s \leq t} V_s$ . In all experiments  $c_t$  is a flat 10 bps fee on every position change. An optional proportional mode scales the cost by  $|\Delta \text{pos}|$  but is not used in this report. The log-return form is numerically stable and scale-invariant across tickers, and the drawdown term directly penalises excursions below the running portfolio peak.

Training episodes sample a random 252-day window from the 2034-day training set and therefore expose the agent to diverse market regimes per episode. Validation and test episodes always start at index 0 and run for the full split length, so the evaluation is deterministic and reproducible. The environment tracks position history, trades and portfolio value, and computes annualised Sharpe ratio, maximum drawdown and trade frequency at episode end.

### 4 Deep Learning Methods

Figure 3 gives an overall view of the DQN training architecture used throughout this work. The online network  $Q_\theta$  interacts with TradingEnv through an  $\varepsilon$ -greedy policy, transitions are pushed to a replay buffer, and mini-batches sampled from the buffer drive a temporal-difference loss against a slowly-updated target network  $Q_{\bar{\theta}}$ .

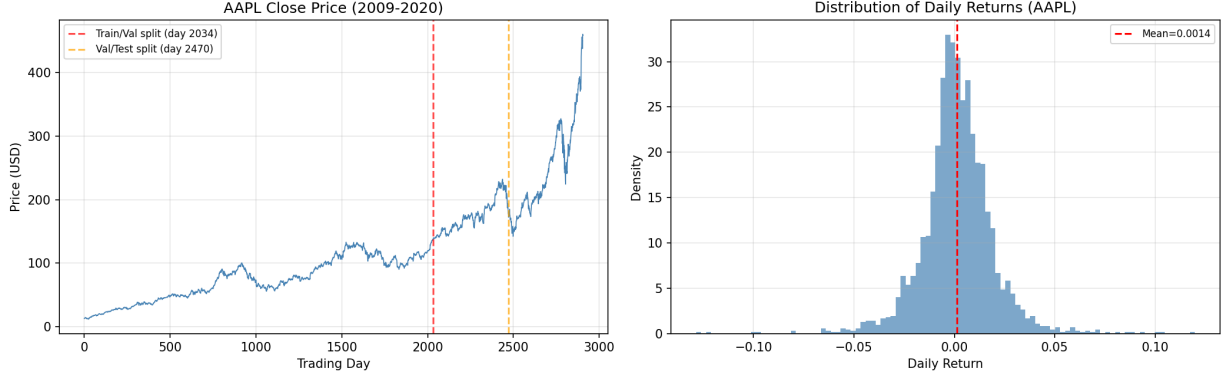


Figure 1: **Left.** AAPL close price from 2009 to 2020 with train, validation and test boundaries. **Right.** Daily return distribution showing the characteristic fat tails of equity returns.

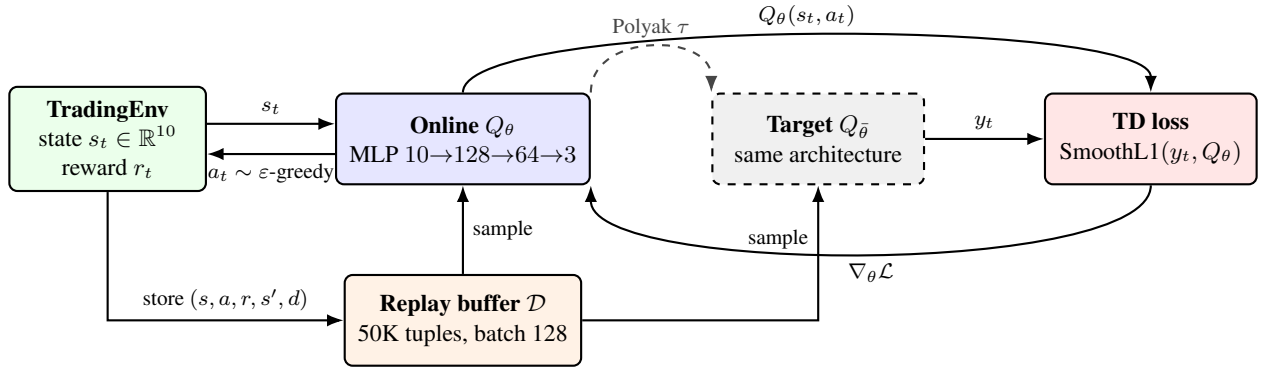


Figure 3: DQN training architecture for our `TradingEnv`. The online network  $Q_\theta$  chooses actions under an  $\epsilon$ -greedy policy, and the environment writes each transition into the replay buffer  $\mathcal{D}$ . Mini-batches sampled from  $\mathcal{D}$  drive a SmoothL1 temporal-difference loss against the bootstrap target  $y_t$  produced by the target network  $Q_{\bar{\theta}}$ , while a Polyak soft update slowly tracks  $Q_\theta$  into  $Q_{\bar{\theta}}$ . DDQN reuses the same diagram and only replaces  $y_t$  by the Double-DQN target of Eq. 5.

**DQN** [2] approximates the optimal action-value function  $Q^*(s, a)$  with a neural network  $Q_\theta$  trained via a Huber (smooth- $\ell_1$ ) temporal-difference loss with a bootstrapped, terminal-masked target:

$$y_t = r_t + \gamma (1 - d_t) \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a'), \quad (3)$$

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} [\text{SmoothL1}(y_t, Q_\theta(s, a))], \quad (4)$$

where  $d_t \in \{0, 1\}$  is the episode-termination flag and  $\mathcal{D}$  is the replay buffer. Key components, all implemented from scratch: a replay buffer [4] of capacity 50K with batch size 128, a soft Polyak target-network update [5] with  $\tau = 5 \times 10^{-3}$  per step, an exponential  $\epsilon$ -decay schedule from 1.0 to 0.02 with decay constant 20K steps, discount factor  $\gamma = 0.99$ , and reward clipping to  $[-1, 1]$  to bound the TD target.

**DDQN** [3] keeps the same loss and replay machinery but decouples action *selection* from action *evaluation* at the target, which empirically reduces the over-estimation bias of vanilla DQN:

$$y_t^{\text{DDQN}} = r_t + \gamma (1 - d_t) Q_{\bar{\theta}}(s_{t+1}, \arg \max_{a'} Q_\theta(s_{t+1}, a')). \quad (5)$$

Our DDQN subclasses DQN and overrides only this target computation. Everything else, including the network, optimiser,  $\epsilon$ -schedule and replay buffer, is shared between the two agents.

**What each component is for.** Re-deriving these algorithms from scratch gave us a concrete understanding of why each ingredient is present. The **replay buffer** breaks the temporal correlation of on-policy samples so that stochastic-gradient updates see ap-

proximately i.i.d. mini-batches. The **target network**  $Q_{\bar{\theta}}$  stabilises bootstrapping by keeping the TD target slowly-changing relative to the online network. A hard copy of the weights caused training oscillations in our early runs and the soft Polyak update smoothed them out. The **exponential  $\varepsilon$ -decay** schedule balances early exploration, where the agent randomly flips its position, against later exploitation, and it kept exploration alive significantly longer than the linear schedule we first tried. **Reward clipping** combined with the Huber loss bounds the TD target and makes large outlier returns such as those around the COVID crash much less destabilising than the square loss. Finally, the **DDQN target decoupling** addresses the observation that  $\max_{a'} Q_{\bar{\theta}}(s', a')$  systematically overshoots the true maximum in noisy function approximation settings.

**Architecture.** Both agents use the same 3-layer MLP:  $10 \rightarrow 128 \rightarrow 64 \rightarrow 3$  with ReLU activations, the Adam optimiser [6] with  $\text{lr} = 5 \times 10^{-4}$ , and gradient-norm clipping at 1.0.

## 5 Baselines

To place the DQN and DDQN agents in context we compare against four classical baselines evaluated on the same 437-day test split. Our primary reference is passive buy-and-hold. We additionally report three simple active baselines covering a linear time-series forecaster, a purely technical rule, and a non-linear supervised-learning model that uses the same feature set as the RL agents.

**Buy-and-Hold (B&H).** The simplest passive strategy buys one unit at the start of the test period and holds it until the end. On AAPL over 2018–2020 it captures the full secular uptrend and is well known to be hard to beat with active trading. We therefore treat it as a reference rather than a target to surpass.

**ARIMA(1,0,0) walk-forward.** At each test step  $t$  we refit an AR(1) model on a rolling 252-day window of recent daily returns, warmed up with the full training and validation history, and we forecast the next return  $\hat{r}_{t+1}$ . The position is +1 if  $\hat{r}_{t+1} > 0$  and 0 otherwise. This baseline is deliberately long-only whereas the RL agent may also go short, which keeps the time-series baseline simple and conservative. It acts as a sanity check on whether a textbook autoregressive signal already extracts useful predictive structure from the data.

**Momentum (20-day).** At each test step we compute

the trailing 20-day cumulative return on the close price. A positive trailing return triggers a long position and a non-positive return leaves the agent flat, again using only the long-and-flat subset of the position space. Momentum is a classic factor in financial economics and serves as a standard weak baseline for active strategies on trending assets.

**Random Forest classifier.** We train a `RandomForestClassifier` with 200 trees on the concatenated training and validation set, using the same 9 z-score-normalised features that the RL agents observe, with a binary label indicating whether the next day’s raw return is positive. On the test split a predicted up day gives a long position and a predicted down day gives a flat position. This baseline asks whether a powerful non-linear supervised learner on the same feature set can already outperform the RL agents.

## 6 Training and Results

Each agent trains for 200 episodes. Each episode samples a random 252-day window from the 2034-day training set in order to expose the agent to diverse market regimes. Validation uses a deterministic greedy rollout from day 0, evaluated every 10 episodes, and the checkpoint with the best validation score is restored for final test evaluation. We rank checkpoints first by cumulative return and then by Sharpe ratio. The left panel of Fig. 4 shows the per-episode training reward as a 20-episode moving average. DQN exhibits a clear upward trend after the initial exploration phase, while DDQN’s reward curve is noticeably noisier, which is consistent with its more conservative target making its greedy policy slower to commit.

A detailed episode walkthrough showing position decisions, cumulative reward, and trade markers is provided in Appendix B (Figure 7).

The test period spans 2018 to 2020 and therefore includes the COVID-19 crash of early 2020, so it is a fairly severe out-of-sample stress test. Reading Table 1 against the stated pedagogical goal, the numbers tell a richer story than a simple agent-vs-buy-and-hold comparison would suggest.

The four classical baselines already span an unusually wide range. Buy-and-hold captures the full  $\sim 3\times$  secular uptrend of AAPL and delivers a very strong raw return of +159%. Simple 20-day price momentum on the same data actually beats buy-and-hold in

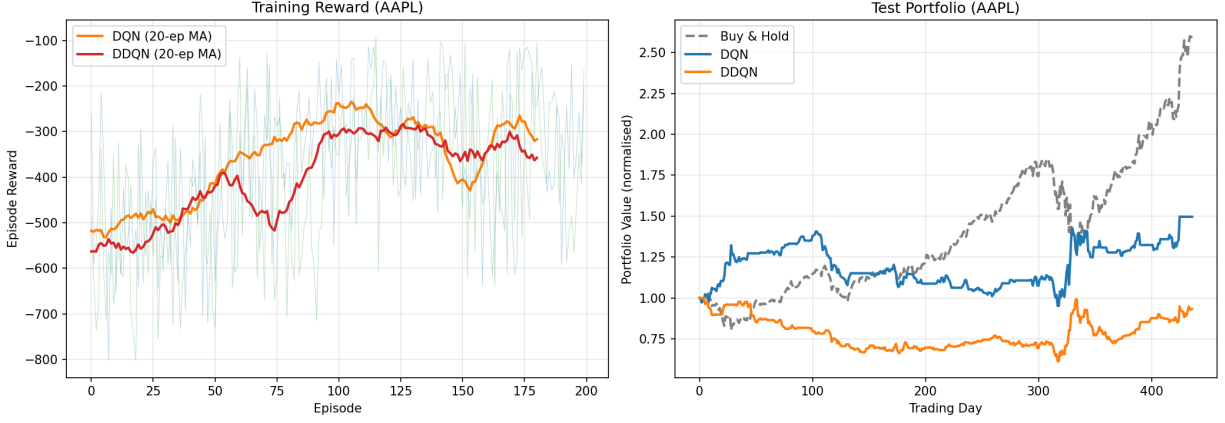


Figure 4: **Left:** Training reward per episode, shown as a 20-episode moving average for DQN and DDQN. **Right:** Test portfolio-value trajectory of DQN, DDQN, the ARIMA(1,0,0) walk-forward baseline and buy-and-hold on AAPL over 2018–2020. The Momentum and Random Forest baselines are reported in Table 1 but are not included in this plot to keep it readable.

Table 1: Test performance on AAPL over the 437-day test split (2018–2020). All active baselines operate in the long-and-flat subset of the position space, while the RL agents also have access to short positions. Best value in each row is shown in bold.

|              | DQN    | DDQN   | B&H     | ARIMA(1,0,0) | Momentum-20     | Random Forest |
|--------------|--------|--------|---------|--------------|-----------------|---------------|
| Cum. Return  | +49.6% | −6.7%  | +159.0% | +121.1%      | + <b>167.0%</b> | +27.1%        |
| Sharpe       | +0.84  | +0.03  | +1.65   | +1.67        | + <b>2.61</b>   | +0.61         |
| Max Drawdown | −32.4% | −38.7% | −31.4%  | −18.8%       | − <b>17.5%</b>  | −31.5%        |

every metric we track, with a +167% cumulative return, a Sharpe above 2.6 and a maximum drawdown of only −17.5%, cut nearly in half relative to the passive benchmark. The ARIMA(1,0,0) walk-forward baseline reaches a comparable Sharpe but a smaller cumulative return of +121%. In striking contrast, the non-linear Random Forest classifier trained on the same 9-feature state ends the test with only +27% and a drawdown almost identical to buy-and-hold, indicating that raw model capacity on this feature set is not what drives good trading performance.

Against this backdrop, DQN and DDQN land in the middle of the baseline ordering rather than at the bottom. DQN finishes with a clearly positive cumulative return of +49.6% and a Sharpe of about 0.84. This places it above the Random Forest classifier on every metric and above DDQN on every metric, yet below both ARIMA and momentum. Its maximum drawdown of −32.4% is slightly worse than that of buy-and-hold at −31.4%, so we cannot claim that the drawdown-shaped reward actually reduced worst-case drawdown

on this split. DDQN ends with a small negative cumulative return and a near-zero Sharpe, consistent with its more conservative target formulation producing systematically smaller  $Q$ -values and, in this regime, translating into missed long positions during the post-crash recovery rather than any visible gain in stability.

Taken together, the experiment illustrates three lessons from first principles. First, the gap between an RL agent and buy-and-hold is not automatic evidence of a hard problem, because simple classical rules like 20-day momentum can also beat buy-and-hold on this particular trending asset. Second, a textbook from-scratch DQN is not a magic replacement for well-understood trading signals, and in this setting it underperforms both ARIMA and momentum while outperforming a Random Forest trained on the same features. Third, the difference between DQN and DDQN is visible not as a simple better-or-worse gap but as a qualitatively different policy. The DQN episode walkthrough in Appendix B (Fig. 7) shows that DQN commits to long positions for extended stretches and only retreats

to flat around the sharpest drawdowns, whereas the DDQN portfolio trajectory in Fig. 4 stays far closer to the zero-position line, matching its more conservative target and producing fewer decisive long or short commitments. Both policies are internally plausible given their respective target formulations, and the goal of the project, namely understanding how textbook DQN variants actually behave on real market data relative to simple classical rules, is met.

## 7 Challenges

An early version of the environment used normalised returns with standard deviation close to one instead of raw returns for rewards, which produced unrealistic values. We fixed this by keeping `raw_return` as a separate column in the dataframe. Initial linear  $\epsilon$ -decay exhausted exploration too quickly and we switched to an exponential schedule to give a smoother exploration to exploitation transition. The MSE loss was sensitive to large reward outliers and was replaced with a Huber smooth- $\ell_1$  loss together with reward clipping to the range  $[-1, 1]$ . Hard target-network copies caused training instability and were replaced by soft Polyak updates with  $\tau \approx 0.005$ . Finally, because shorting can in principle drive portfolio value below zero, we clamp the compounded portfolio at  $10^{-8}$  for numerical safety.

## 8 Future Work

Natural extensions of this project include policy-gradient methods such as PPO for continuous position sizing, multi-asset portfolio environments that trade several DJIA tickers jointly, recurrent LSTM architectures for explicit temporal dependencies, and reward formulations based directly on Sharpe or CVaR rather than raw log-returns.

## 9 Contributions

### Yufan.

- Initial from-scratch implementation of the entire codebase.
- Data pipeline with Kaggle `alincijov/trading` via `kagglehub` and 9-feature engineering (RSI, MACD, Bollinger, ATR, OBV, MA deviations, normalised volume).

- Gym-style `TradingEnv` with drawdown-penalised reward, transaction cost models, portfolio tracking and Sharpe / drawdown / trade-frequency evaluation.
- Initial DQN and DDQN agent with `QNetwork`, replay buffer, target network and training loop.
- Random Forest baseline (200 trees on the same feature set).
- Data pipeline and training figures, appendix material, and most of the report writing.

### Rutwik.

- Refinements to the agent and environment: position-delta action space, Huber loss, reward clipping and log-return reward formulation.
- Exponential  $\epsilon$ -decay and soft Polyak target-network updates.
- Random-window training episodes and best-model checkpointing.
- Hyperparameter retuning (learning rate, batch size, buffer capacity).
- Classical baselines: buy-and-hold, ARIMA(1,0,0) walk-forward and 20-day momentum, integrated into the reporting and plotting flow.

## References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *AAAI*, 2016, pp. 2094–2100.
- [4] L.-J. Lin, “Self-improving reactive agents based on reinforcement learning, planning and teaching,” *Machine Learning*, vol. 8, no. 3–4, pp. 293–321, 1992.
- [5] B. T. Polyak and A. B. Juditsky, “Acceleration of stochastic approximation by averaging,” *SIAM Journal on Control and Optimization*, vol. 30, no. 4, pp. 838–855, 1992.

- [6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *ICLR*, 2015.
- [7] G. Brockman, V. Cheung, L. Pettersson, et al., “OpenAI Gym,” *arXiv preprint arXiv:1606.01540*, 2016.
- [8] A. Paszke, S. Gross, F. Massa, et al., “PyTorch: An imperative style, high-performance deep learning library,” in *NeurIPS*, 2019, pp. 8024–8035.
- [9] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Transactions on Neural Networks*, vol. 12, no. 4, pp. 875–889, 2001.
- [10] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, “Deep direct reinforcement learning for financial signal representation and trading,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 3, pp. 653–664, 2016.
- [11] Z. Jiang, D. Xu, and J. Liang, “A deep reinforcement learning framework for the financial portfolio management problem,” *arXiv preprint arXiv:1706.10059*, 2017.
- [12] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, “Deep reinforcement learning for automated stock trading: An ensemble strategy,” in *ACM ICAIF*, 2020.

## A Data Pipeline Visualisations

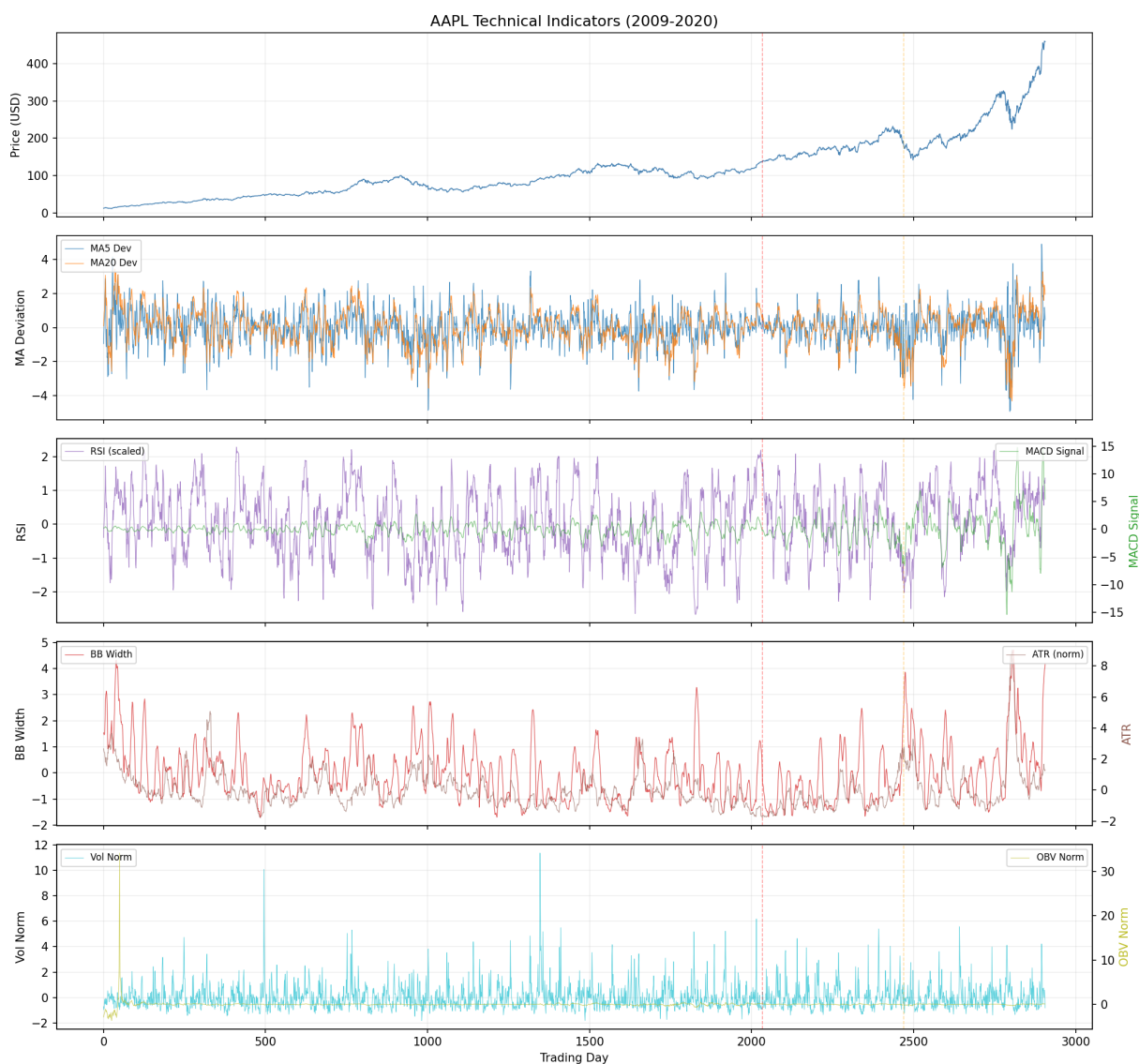


Figure 5: Technical indicator time series for AAPL (2009–2020). From top: close price, MA deviations, RSI & MACD, Bollinger width & ATR, volume & OBV. Dashed lines mark train/val/test boundaries.

Feature Distributions: Train vs Val vs Test (z-score normalised, AAPL)

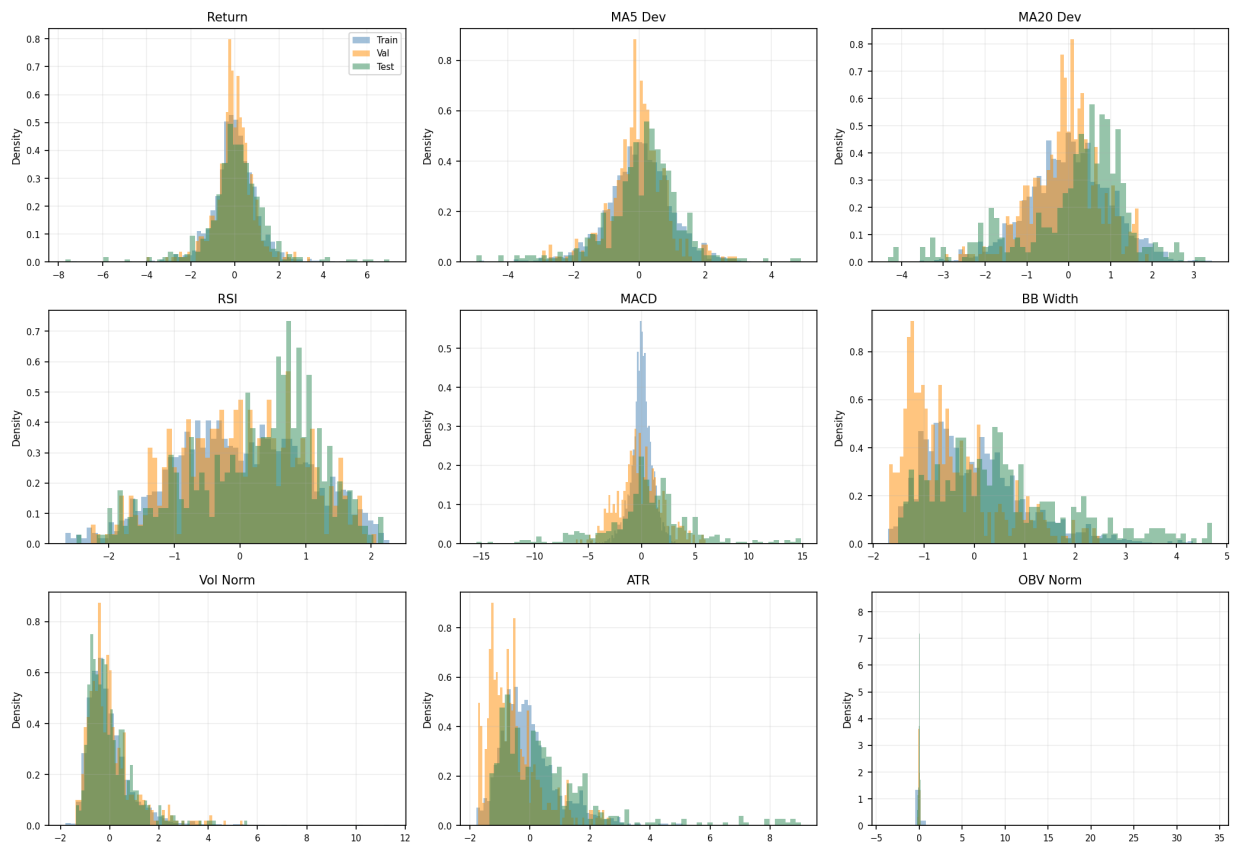


Figure 6: Feature distributions across train/val/test splits (z-score normalised using training-set statistics only). Overlapping distributions confirm consistent normalisation without look-ahead bias.

## B Episode Walkthrough

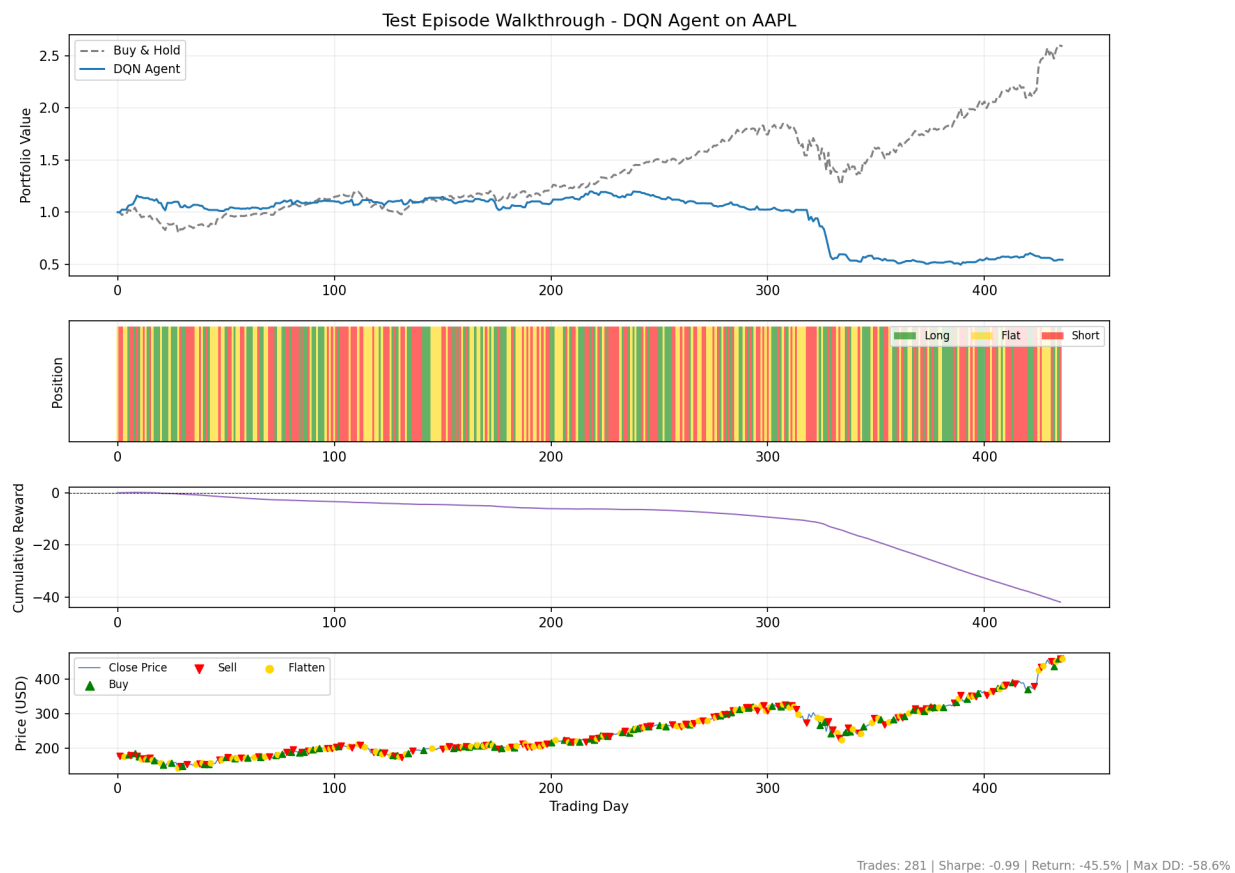


Figure 7: DQN test episode walkthrough on AAPL. Top: portfolio value vs buy-and-hold. Second: position colour bands (green=long, gold=flat, red=short). Third: cumulative reward. Bottom: buy/sell markers on close price.